PART I

Power Iteration: Implementation and Numerical Experiments

Abstract. This report describes the Power Iteration component of the project. The method estimates an eigenvalue of largest magnitude and its unit eigenvector. The implementation uses the Rayleigh quotient for the eigenvalue estimate and stops when the eigenpair residual is below a chosen tolerance. Experiments compare results with NumPy and study the effect of the spectral ratio on convergence.

1 Objective and method

Given a square matrix A and a nonzero starting vector x_0, Power Iteration repeatedly multiplies by A and normalizes:

$$x_{k+1} = A\,x_k \,/\, \|A\,x_k\|_2, \quad \hat{\lambda}_{k+1} = (x_{k+1})^{*}\,A\,x_{k+1}.$$

The second expression is the Rayleigh quotient (with x^{*} denoting the conjugate transpose). The stopping criterion is the eigenpair residual

$$r_{k+1} = \|A\,x_{k+1} - \hat{\lambda}_{k+1}\,x_{k+1}\|_2.$$

The code stops when $r_{k+1} <$ tol; otherwise it continues up to `max_iter`. The experiments use tol $= 10^{-8}$ and `max_iter` $= 10000$. The returned history stores $(\hat{\lambda}_k, r_k)$ at each iteration.

2 Implementation and validation

The `PowerIteration` class in `eigen_lab.py` checks that the matrix is square and that the starting vector has the correct dimension and is nonzero. It normalizes the vector, computes the Rayleigh quotient using `numpy.vdot`, and evaluates the residual using the Euclidean norm. It returns the eigenvalue estimate, unit eigenvector and history. It raises `RuntimeError` if the tolerance is not reached, and gives a `UserWarning` if the matrix annihilates the current vector.

The uploaded `test.py` checks the default matrix $A = \mathrm{diag}(5, 2)$ with $x_0 = (1, 1)^{\mathrm{T}}$. It compares the computed eigenvalue with the eigenvalue of largest magnitude from `numpy.linalg.eigvals`; the test passes when the residual is below the tolerance and the eigenvalue error is below 10 tol.

3 Experiment 1: a clearly dominant eigenvalue

The main experiment uses the symmetric matrix $A = \mathrm{diag}(5, 2.5, 1)$, $x_0 = (1, 1, 1)^{\mathrm{T}}$. Its eigenvalues are 5, 2.5 and 1. Thus $\lambda_1 = 5$ is dominant and $|\lambda_2/\lambda_1| = 0.5$. NumPy gives the reference eigenvalue 5. Power Iteration returns 5.0 after 28 iterations, with final residual 9.31×10^{-9}. The computed unit eigenvector is approximately $(1, 3.73 \times 10^{-9}, 2.68 \times 10^{-20})^{\mathrm{T}}$, close to $(1, 0, 0)^{\mathrm{T}}$.

Table 1 lists selected Rayleigh quotient estimates from the iteration history; the error is measured against the reference eigenvalue 5. Figure 1 shows that the eigenvalue estimate approaches the NumPy reference value while the residual decreases.

k	$\hat{\lambda}_k$	$\lvert\hat{\lambda}_k - 5\rvert$	r_k
1	4.3914729	6.09×10^{-1}	1.16
5	4.9975606	2.44×10^{-3}	7.81×10^{-2}
10	4.9999976	2.38×10^{-6}	2.44×10^{-3}
15	5.0000000	2.33×10^{-9}	7.63×10^{-5}
28	5.0000000	0 (rounded)	9.31×10^{-9}

Table 1: *Selected eigenvalue estimates and residuals for the main experiment.*

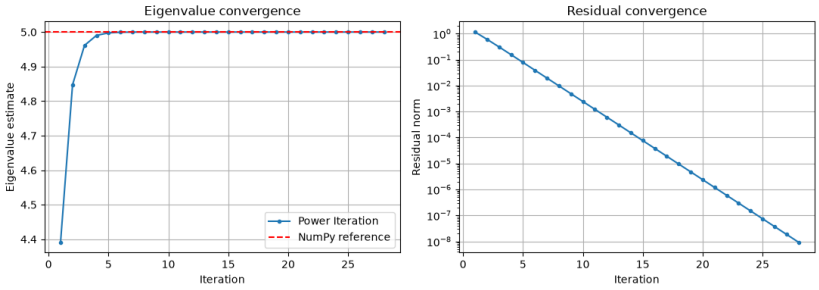


Figure 1: *Eigenvalue and residual convergence for the main experiment.*

4 Experiment 2: effect of the spectral ratio

To study convergence speed, the experiment uses $A_r = \mathrm{diag}(1, r)$ and $x_0 = (1, 1)^T$. Here the dominant eigenvalue is 1 and $|\lambda_2/\lambda_1| = r$. Each run uses tolerance 10^{-8}. Figure 2 compares residual convergence for three spectral ratios.

Ratio r	Iterations	Final residual	Result
0.50	26	7.45×10^{-9}	Fast
0.90	153	9.98×10^{-9}	Slower
0.99	1375	9.96×10^{-9}	Much slower

Table 2: *Iteration counts for different spectral ratios.*

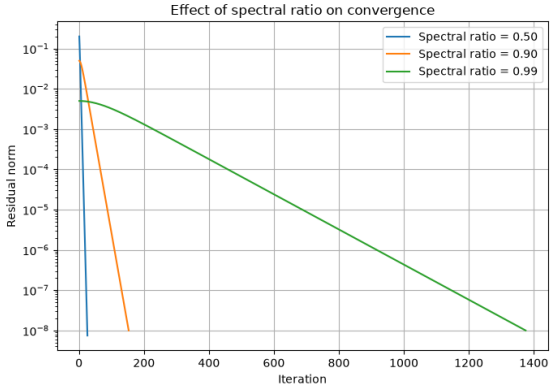


Figure 2: *Residual for different spectral ratios.*

The results show the expected trend: when a unique dominant eigenvalue exists, the convergence rate is governed by the ratio $|\lambda_2/\lambda_1|$. As this ratio approaches 1, the two largest eigenvalue magnitudes become less separated and more iterations are required. In these runs, the count increases from 26 to 1375 as the ratio rises from 0.50 to 0.99.

5 Limitations and conclusion

Power Iteration works best when one eigenvalue has strictly larger magnitude than all others and the starting vector has a component in its eigendirection. If two eigenvalues share the largest magnitude (for example, the eigenvalues $+i$ and $-i$ of a rotation matrix), the iterates may oscillate and the residual may not reach the tolerance. If the starting vector lies in the null space of a nonzero matrix, the implementation warns that it has been annihilated and returns a zero estimate; this is not the dominant eigenpair. These behaviours are described in the project `README`. The currently uploaded `test.py` automates the default diagonal test; it does not yet automate separate complex-eigenvalue or null-space cases.

Overall, the implementation approximates the dominant eigenpair for a matrix with a clear dominant eigenvalue, uses a residual-based stopping rule, and agrees with NumPy in the main experiment. The spectral-ratio experiment confirms that convergence becomes much slower as the ratio approaches one.

PART II

Classical and Modified Gram–Schmidt QR Factorization: Numerical Experiments in eigenlab

1 Methods and error measures

The class `QRFactorization` factors an $m \times n$ matrix A ($m \geq n$) as $A = QR$, with $Q \in \mathbb{R}^{m \times n}$ having orthonormal columns and $R \in \mathbb{R}^{n \times n}$ upper triangular. Three variants were tested: classical Gram–Schmidt (CGS), modified Gram–Schmidt (MGS), and MGS with one extra orthogonalization pass (MGS+reorth.). The extra pass is run only if the first-pass orthogonality error exceeds tol. A column is declared (nearly) linearly dependent, and a `ValueError` is raised, when the norm of its remaining component falls below tol. Each factorization is scored with three Frobenius-norm errors:

$$\text{reconstruction} = \|A - QR\|_F, \quad \text{orthogonality} = \|Q^T Q - I\|_F, \quad \text{triangularity} = \|\text{tril}(Q^T A, -1)\|_F.$$

The last quantity is called “triangularity” in the code. It measures how much of column j of A lies along a later q_i ($i > j$), which is exactly zero in exact arithmetic.

2 Experiments

Experiment 1 factors five random 3×3 matrices (uniform entries, seed 2026) with MGS and tol = 10^{-12}. **Experiment 2** passes the rank-deficient matrix [[1, 2, 3], [4, 5, 6], [5, 7, 9]] (column 3 = 2·column 2 − column 1) to MGS with the default tol = 10^{-8}. **Experiment 3** factors the 7×7 Hilbert matrix H_7 ($h_{ij} = 1/(i + j - 1)$, condition number about 4.8×10^8) with CGS, MGS, and MGS+reorth., all with tol = 10^{-12}.

3 Results

Experiment 1. All five factorizations are accurate to near machine precision (Table 3).

Error	Matrix 1	Matrix 2	Matrix 3	Matrix 4	Matrix 5
Orthogonality	6.92×10^{-16}	8.97×10^{-16}	1.39×10^{-16}	8.70×10^{-16}	2.24×10^{-14}
Reconstruction	1.11×10^{-16}	0	2.78×10^{-17}	1.11×10^{-16}	1.11×10^{-16}
Triangularity	4.86×10^{-16}	6.04×10^{-16}	8.78×10^{-16}	4.38×10^{-16}	1.73×10^{-14}

Table 3: *Experiment 1: MGS on five random 3×3 matrices.*

Experiment 2. The `ValueError` was raised, so the rank deficiency was detected.

Experiment 3. None of the three methods raised an error at tol = 10^{-12} (Table 4).

Method	Orthogonality	Reconstruction	Triangularity
CGS	7.98×10^{-1}	2.23×10^{-6}	9.91×10^{-7}
MGS	8.08×10^{-9}	8.67×10^{-17}	8.55×10^{-9}
MGS + reorth.	4.87×10^{-16}	9.41×10^{-17}	1.70×10^{-16}

Table 4: *Experiment 3: factorization errors for the 7×7 Hilbert matrix.*

4 Interpretation

- **Well-conditioned input is easy.** On the random 3×3 matrices every error is at most 2×10^{-14}, so the factorization is accurate and the rank test does not produce false alarms. Only MGS was run in this experiment, so it does not compare the methods.
- **CGS breaks down on H_7.** Its orthogonality error is 0.81, so Q is not even approximately orthonormal, and its reconstruction error (2.3×10^{-6}) is about 4×10^{10} times larger than MGS’s.
- **MGS is far better but still loses orthogonality.** Its orthogonality error is smaller than CGS’s by a factor of about 6×10^7, yet at 1.25×10^{-8} it is still far from machine precision.
- **Small reconstruction error does not certify Q.** For MGS the reconstruction error is 5.6×10^{-17} while the orthogonality error is 1.3×10^{-8}. Orthogonality must therefore be checked separately.
- **One extra pass restores orthogonality.** Because $1.25 \times 10^{-8} > 10^{-12}$, the second pass was triggered, and it reduced all three errors to about 10^{-16}.

5 Conclusions and limitations

Modified Gram–Schmidt should be preferred to classical Gram–Schmidt, and reorthogonalization is worthwhile when the first-pass orthogonality error is above roundoff, as for H_7. These conclusions rest on a single ill-conditioned matrix ($n = 7$), a single random seed, and 3×3 random matrices run with MGS only. They do not show how the errors scale with conditioning or size. Dependence detection uses an absolute tolerance, so its behavior depends on the scale of the data.

Theoretically, both MGS and GS are not as backward stable as other QR factorization methods such as the Householder method. Reorthogonalization just once is sufficient to restore orthogonality to matrix Q for most cases. Numerical rank also determines the success of our code because if the numerical rank is less than the rank of the matrix, then the columns will look linearly independent to the computer.

PART III

QR Iteration Experiments: Short Interpretation

1 Method and setup

QR iteration ($A_{k+1} = R_k Q_k$, where $A_k = Q_k R_k$) was run for up to 1000 iterations with tolerance 10^{-8}. Each plot shows the diagonal entries of A_k against the NumPy eigenvalues (red dashed lines).

2 Experiments and results

2.1 Experiment 1: 2×2 (symmetric)

Matrix: [[2, 1], [1, 3]]

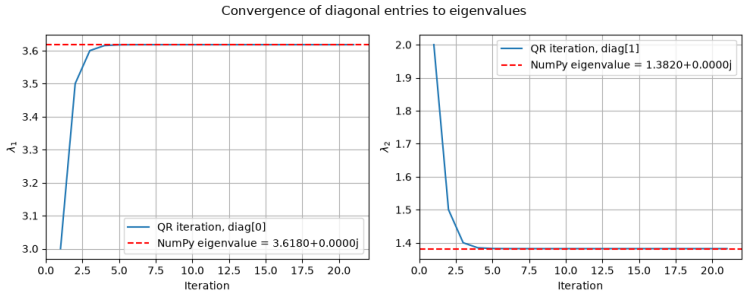


Figure 3: *Experiment 1: convergence of diagonal entries to eigenvalues.*

Eigenvalues 3.618 and 1.382. Large gap (ratio 0.38), so the diagonal settles in about 5 iterations and the run ends in about 21.

2.2 Experiment 2: 3×3 (symmetric)

Matrix: [[4, 1, 0], [1, 3, 1], [0, 1, 2]]

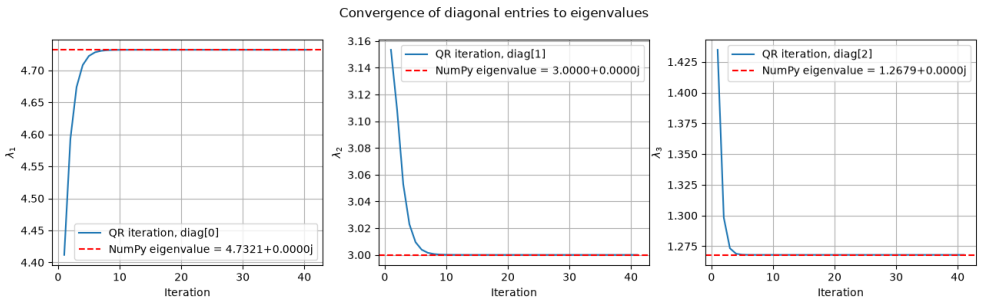


Figure 4: *Experiment 2: convergence of diagonal entries to eigenvalues.*

Eigenvalues 4.732, 3.000, 1.268. The closest pair (4.732 and 3.000, ratio 0.63) sets the run length at about 41 iterations. Convergence is smooth.

2.3 Experiment 3: 4×4 (symmetric)

Matrix: tridiagonal [[2, −1, 0, 0], [−1, 2, −1, 0], [0, −1, 2, −1], [0, 0, −1, 2]]

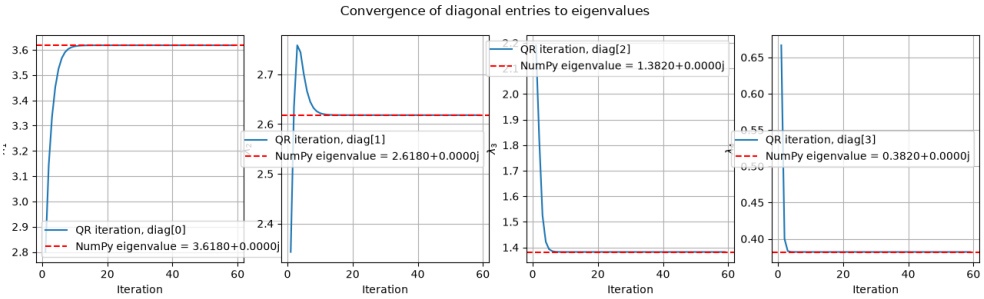


Figure 5: *Experiment 3: convergence of diagonal entries to eigenvalues.*

Eigenvalues 3.618, 2.618, 1.382, 0.382. The close pair 3.618 and 2.618 (ratio 0.72) makes this the slowest symmetric case, at about 60 iterations.

2.4 Experiment 4: 3×3 (non-symmetric, real eigenvalues)

Matrix: [[4, 1, 0], [2, 3, 1], [0, 1, 1]]

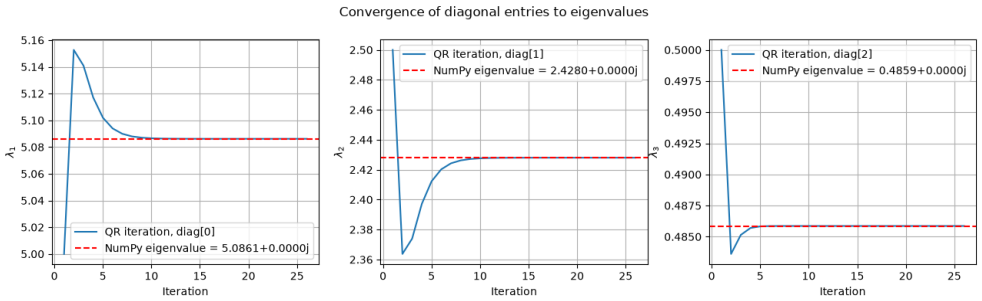


Figure 6: *Experiment 4: convergence of diagonal entries to eigenvalues.*

Eigenvalues 5.086, 2.428, 0.486, found in about 26 iterations, with an early overshoot. The limit is upper triangular, not diagonal: the diagonal gives the eigenvalues, but the columns of Q are Schur vectors, not eigenvectors.

2.5 Experiment 5: 3×3 (non-symmetric, complex eigenvalues)

Matrix: [[1, −2, 0], [2, 1, 0], [1, 0, 3]]

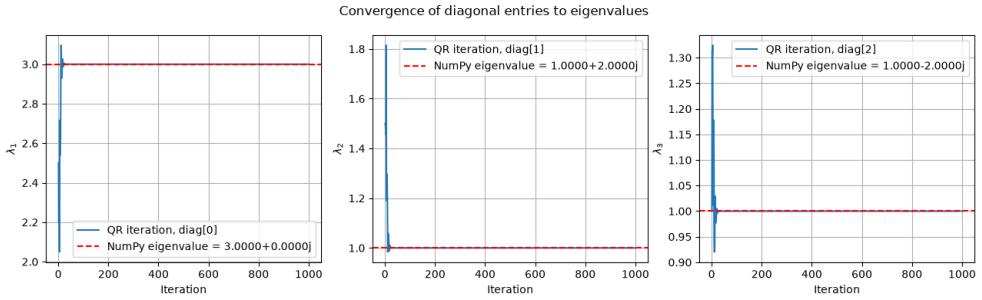


Figure 7: *Experiment 5: convergence of diagonal entries to eigenvalues.*

The eigenvalues are 3 and 1 ± 2i. The first diagonal entry settles at 3, which is correct, but the iteration fails to converge to the complex eigenvalues. The other two both settle at 1, because the diagonal can only show the real part of a complex number. The imaginary part (2) sits in the off-diagonal entries of the small 2×2 block, so the diagonal never shows it. The two complex eigenvalues are the same size, so QR can't split them apart and the block never becomes triangular. That's why the code kept running until it hit 1000 iterations. The method didn't blow up, it just doesn't show complex eigenvalues on the diagonal.

3 Limitations

- **Close eigenvalues:** convergence rate is $|\lambda_{i+1}/\lambda_i|$; ratios near 1 (Experiment 3) are slow, and equal moduli never separate.
- **Diagonal is not always enough:** the diagonal reveals all eigenvalues only when the limit is diagonal (symmetric) or triangular (real eigenvalues). It does not give eigenvectors for non-symmetric matrices.
- **Complex eigenvalues:** a conjugate pair stays as a 2×2 block; read $\lambda = a \pm i\sqrt{(-bc)}$ from the block instead of the diagonal.